

I H A T E T O B D G E T

**A simple web app to understand and control your expenses.
Designed to be self-hosted.**

Installation Guide

1. About This Guide

Greetings, this is the installation guide for the web application IHateToBudget. This is a detailed document that provides guidance on the setup and initial configuration of the system and the underlying ecosystem structure of IHateToBudget—a lightweight, privacy-focused expense management tool built on the Python Django framework.

This installation guide abstracts operational complexities and outlines deterministic workflows for the following procedures:

- **Environment Setup:** Creation of environment boundaries such as Docker Runtimes, Compose Environments, and Python Sandboxes using Pipenv management.
- **Application Bootstrapping:** Setting up environment parameters from sample templates, migration of schema, initialization of database persistence layer, and super user bootstrapping.
- **Localization & Currency Configuration:** Adjusting localization parameters, customizing the prefix and suffix of currency, and mathematical group breaks for localization of the display with any international currency.
- **Maintenance Operations:** Interaction with automated maintenance activities such as cron jobs, backing up the system, upgrading the schema during image upgrade, and running automated test suites.

2. Prerequisites

- **Minimum Hardware Requirements:**

- **Processor:** Single-core CPU (x86_64 or ARM64, such as a Raspberry Pi 3 or higher)
- **Memory:** 1 GB RAM minimum
- **Storage:** 2 GB of available persistent disk space (to accommodate base images, dependencies, and database growth)

- **Containerized Infrastructure Requirements**

If you are deploying the application via container orchestration, the following dependencies must be installed and active on the host machine:

- **Docker Engine:** Version 20.10.0 or higher.
- **Docker Compose:** Version 2.0.0 or higher (integrated or standalone binary).
- **Network Connectivity:** Unrestricted outbound internet access during deployment to pull images from Docker Hub.

3. Step-by-Step Installation

- **Step 1: Clone the Repository**

Initialize a terminal session and clone the repository to your target local directory. Navigate into the root directory of the project:

```
git clone https://github.com/ImY11/ihatetobudget.git
cd ihatetobudget
```

- **Step 2: Environment Configuration**

The application relies on environment variables for security and operational routing. Populate your local configuration from the provided sample blueprint:

- Locate the Docker environment template named `docker-compose.env.example` in the project root. Copy and rename this file to `docker-compose.env`:

```
cp docker-compose.env.example docker-compose.env
```

- Locate the Caddyfile named `Caddyfile.example` in the project root. Copy and rename this file to `Caddyfile` (with no extension):

```
cp Caddyfile.example Caddyfile
```

- Open `docker-compose.env` in a text editor and update critical values, including the `DJANGO_SECRET_KEY` and localization preferences if required.

```
DJANGO_SECRET_KEY=change-me
DJANGO_DEBUG=False
CURRENCY_GROUP_SEPARATOR=change-me
CURRENCY_DECIMAL_SEPARATOR=change-me
CURRENCY_PREFIX=change-me
CURRENCY_SUFFIX=change-me
```

- **Step 3: Launch the Container Stack**

Instruct Docker Compose to pull necessary images, build local containers, and initialize the application stack in detached (background) mode:

```
docker compose up -d --build
```

Ensure your Docker Desktop engine is running on your device.

- **Step 4: Execute Database Migrations**

With the containers running, initialize the relational database schema by running Django's migration engine within the active application container:

```
docker compose exec python manage.py migrate
```

- **Step 5: Create a Super User**

To access the backend configuration panel and initialize your financial tracking profile, provision an application superuser:

```
docker compose exec python manage.py createsuperuser
```

- **Step 6: Access the Application**

Open your preferred web browser and navigate to the application instance interface: <http://127.0.0.1/>

4. Troubleshooting & Logs

- **Accessing Application Logs**

If the application fails to launch or encounters runtime errors, inspect the live container logs using the following command:

```
docker compose logs -f web
```

- **Common Issues and Resolutions**

- **"port 80 is already allocated"**
 - **Cause:** Another web server (like Nginx, Apache, or another Docker container) is occupying the HTTP port.
 - **Resolution:** Open docker-compose.yml and map the host port to an open port (e.g., change 80:80 to 8080:80).
- **"no such service: web"**
 - **Cause:** The service name in your command execution does not match the service block name specified in the configuration files.
 - **Resolution:** Verify the core service name within docker-compose.yml and replace web with the exact service identifier.

5. How to Update the App

When updates or patches are pulled from the remote repository fork, follow this workflow to apply changes safely without data loss:

1. Stop the environment:

```
docker compose down
```

2. Pull the latest codebase revisions:

```
git pull origin master
```

3. Rebuild the container images:

```
docker compose build
```

4. Apply new database migrations if applicable:

```
docker compose run --rm web python manage.py migrate
```

5. Bring the stack back online:

```
docker compose up -d
```